pulsehub.

Supervision locale de télémétrie UDP

Projet individuel
Master Tech-Lead / CTO

Version v1.0

Le besoin d’une régie

Repérer un appareil en défaut
et vérifier la santé de la réception.

50
appareils simulés

20 Hz
par appareil

1 000
trames/s nominales

Vraies trames UDP locales. Aucun appareil physique dans cette démo.

Parcours d’une trame

Étape	Responsabilité	Échange
Simulation	Encoder et injecter les défauts	UDP 9000
Décodage	Valider la structure binaire	Buffer / Frame
Séquencement	Écarter les répétitions et retards	Décision par appareil
Synthèse	Calculer les états et agrégats	Fleet + chaîne JSON
Affichage	Présenter le dernier état accepté	SSE web / HTTP CLI

TypeScript strict. Calcul métier séparé des transports et des interfaces.

Décodage du protocole

16 + 3 × count

octets par datagramme

Big-endian
512 octets maximum
Valeurs int16 signées

Mesure	Valeur brute	Affichage
Température	4250	42,5 °C
Tension	24000	24 V
Courant	3000	3 A

Chaque erreur de structure devient un rejet compté.

Séquences et pertes estimées

Séquence reçue	Décision	Dernière acceptée	Pertes cumulées
10	Accepter la première trame	10	0
12	Accepter et compter un trou	12	1
12	Écarter le doublon	12	1
11	Écarter le retard	12	1

delta = (seq - lastSeq) >>> 0

Le wrap uint32 conserve l’ordre.

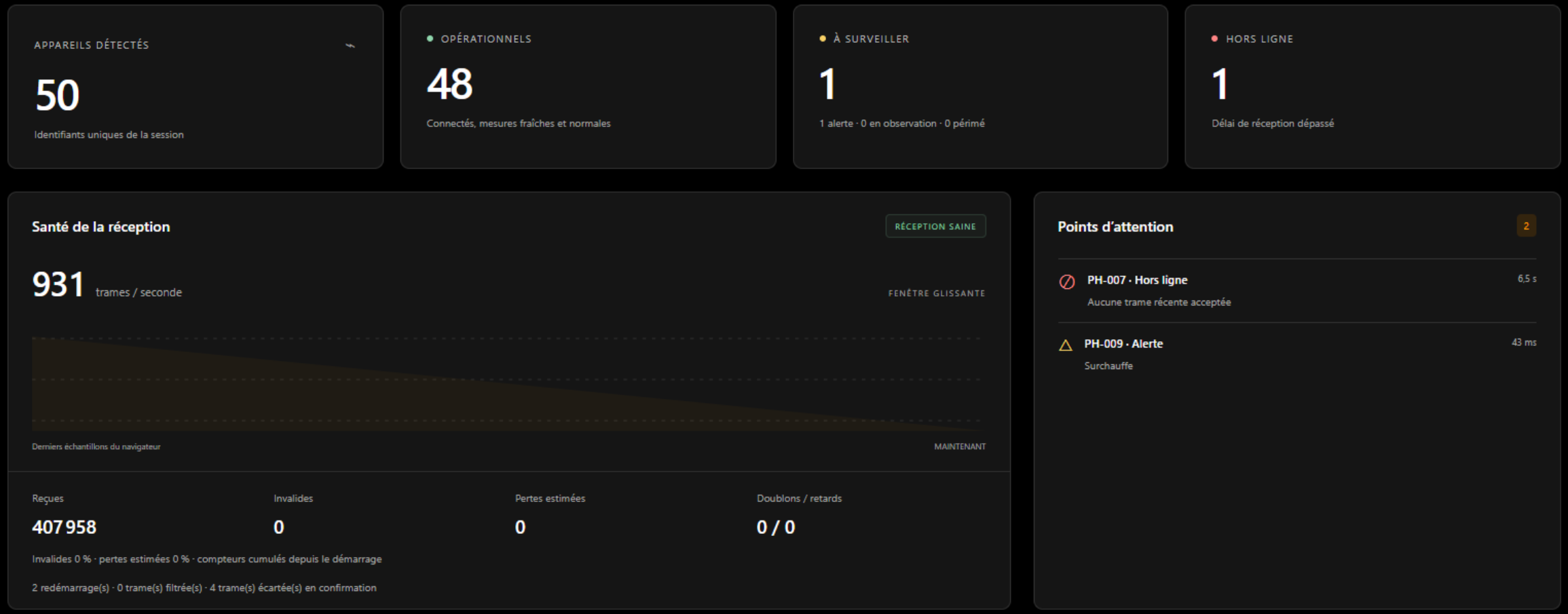
Présence, fraîcheur et durée d’alerte

Situation	Condition par défaut	État affiché
Température haute	> 70 °C depuis moins de 3 s	Observation
Dépassement continu	> 70 °C pendant 3 s	Alerte
Mesure non actualisée	Après 2 s	Mesures périmées
Aucune trame acceptée	Après 3 s	Hors ligne

Un heartbeat confirme la présence, sans rafraîchir les mesures.

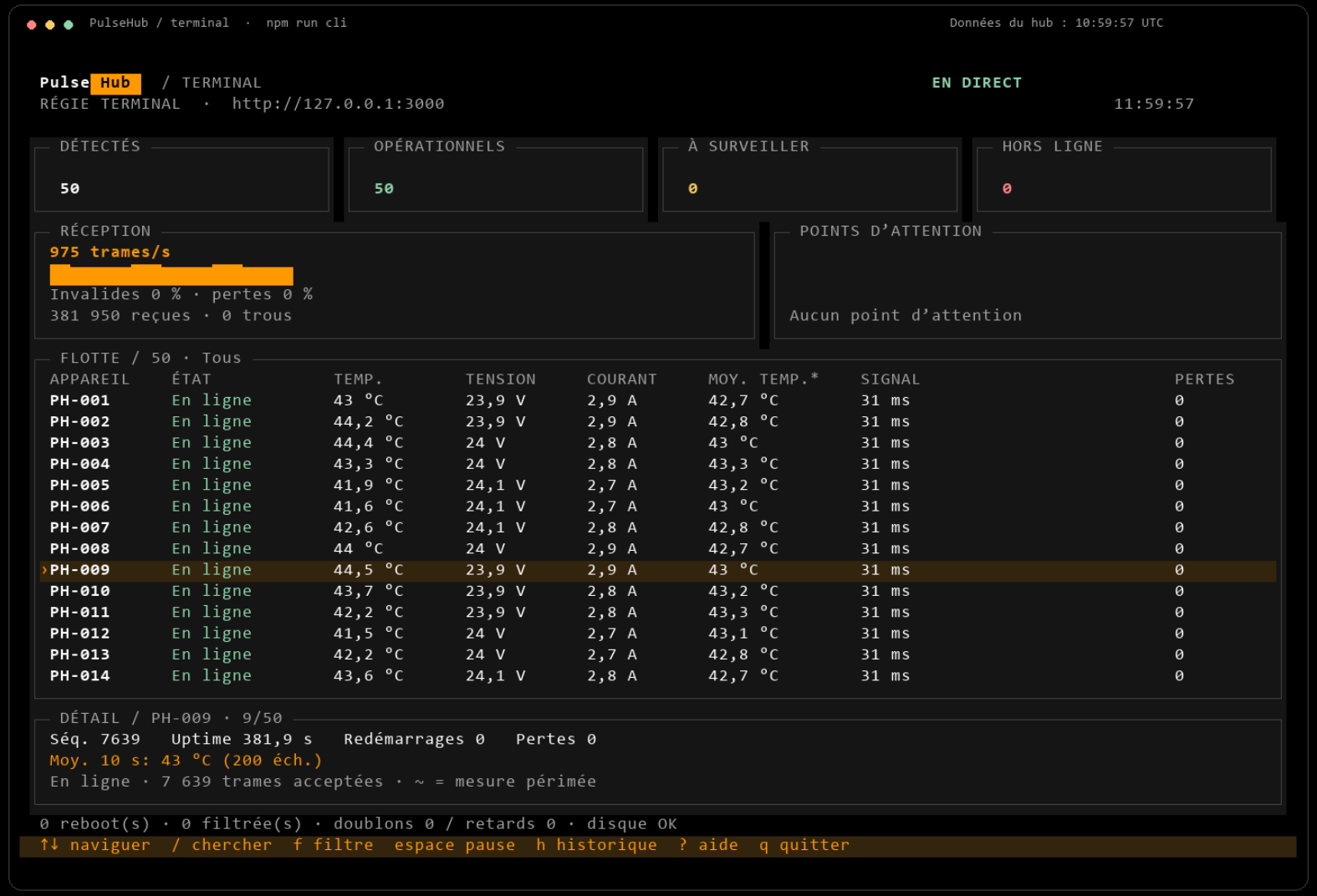
Dashboard web

Vue globale en direct, recherche par appareil et historique consultable



Capture réelle de la démo locale

Interface CLI



Même hub
Même synthèse

/ rechercher
f filtrer
Espace pause
h historique

npm run cli

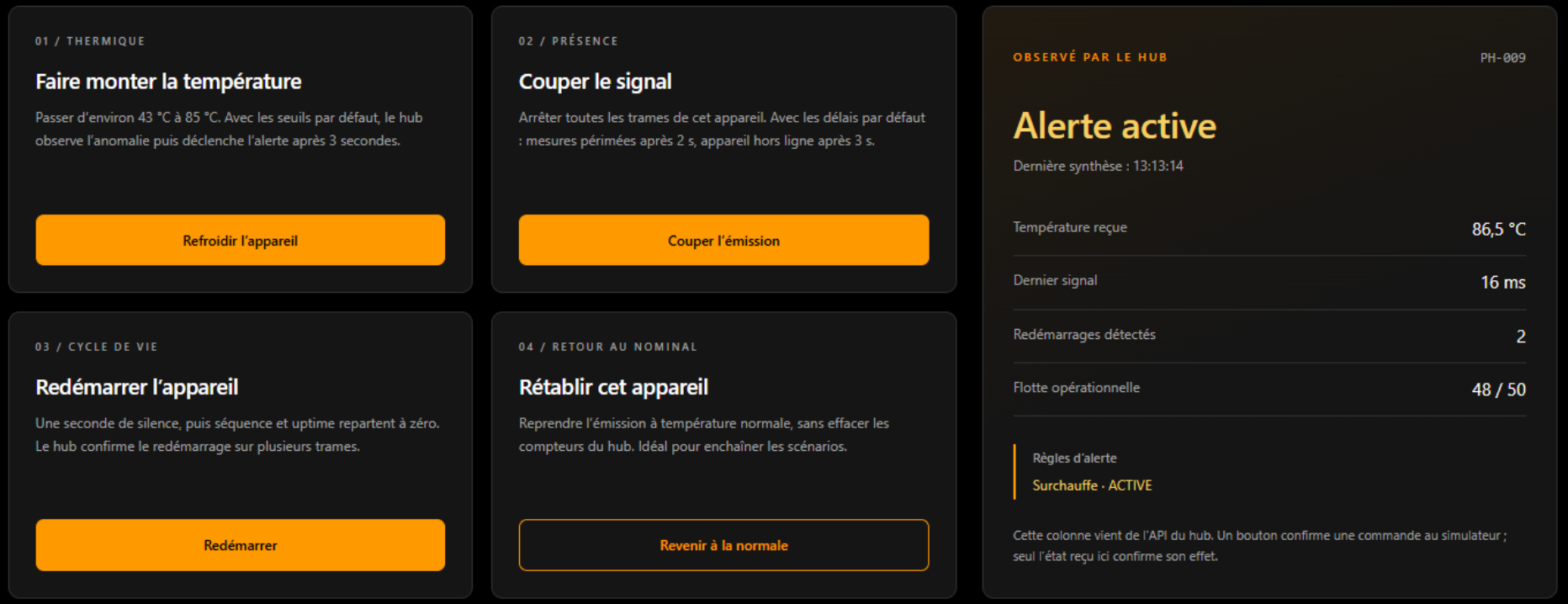
Démonstration pilotée

01 Surchauffe

02 Coupure

03 Reprise

04 Redémarrage



Le panneau observe le hub après l’envoi des vraies trames UDP.

Bonus et validation

43

tests réussis

Lint, typage strict
et compilation
CI Linux et Windows

01 Chaîne JSON extensible

02 Alertes temporisées

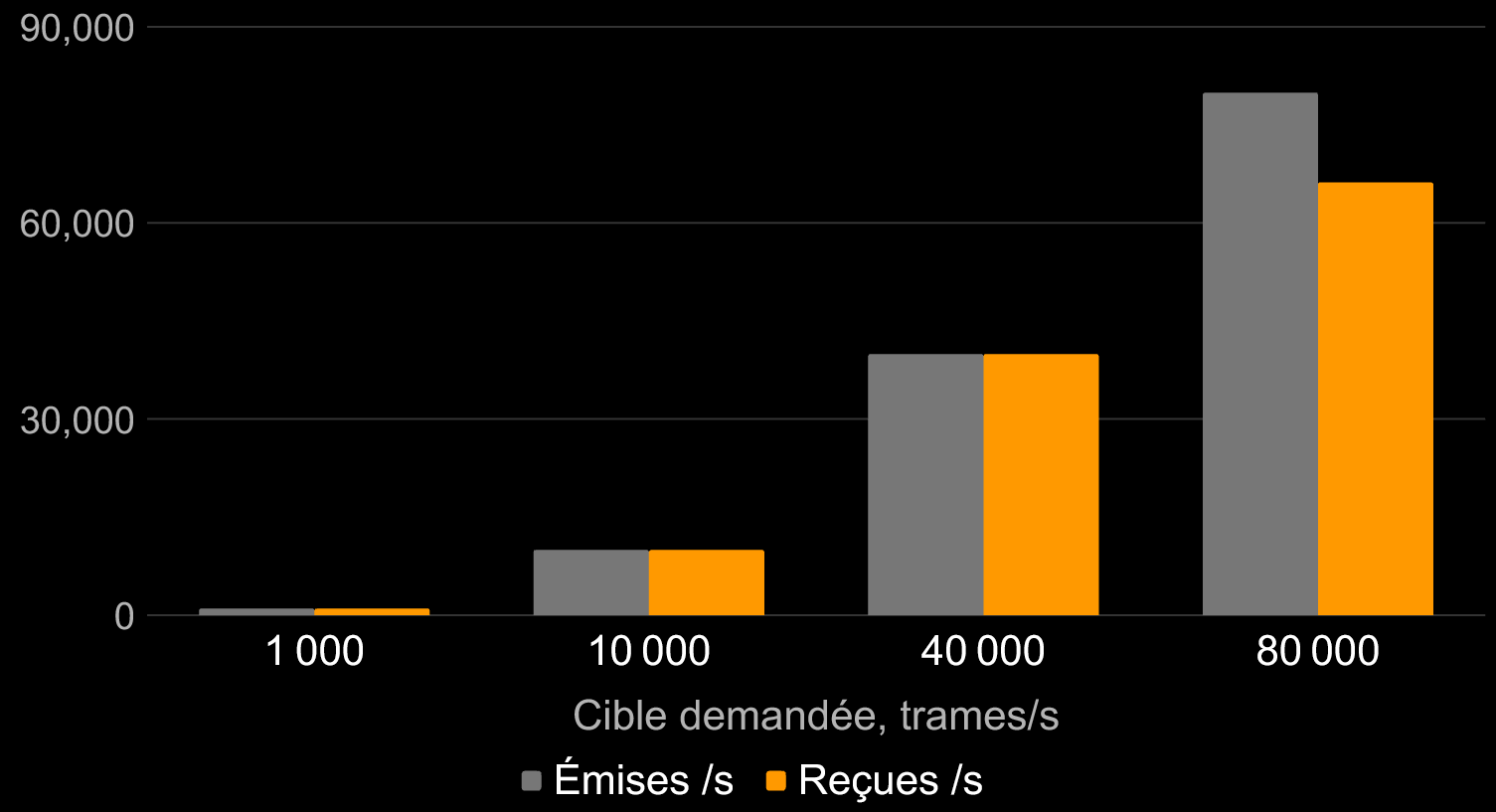
03 Historique persistant

04 Wrap et redémarrage

05 Mesure de débit

06 Intégration continue

Débit mesuré sur cette machine



40 000

trames/s demandées
Plus haut palier validé

332 041 /s

Calcul métier isolé
UDP, HTTP et disque exclus

Windows 11, Ryzen 9 5900HS. Quatre émetteurs, 10 s par palier, UDP local.
Mesures du 30/10/2026. À 80 000 trames/s demandées : 82,83 % de livraison.

Limites et responsabilité

Redémarrages

Heuristique confirmée sur plusieurs trames, sans bootId.

Exploitation

Réseau local de confiance. Historique périodique et borné.

Assistance IA déclarée

Codex a contribué à l’architecture, au code, aux tests et à la documentation.
Chaque ligne livrée doit pouvoir être expliquée.

github.com/sebgrtd/rattrapage

Rendu : v1.0

Trame binaire (annexe A)

Offset	Taille	Champ
0	2	magic = 0x50 0x48
2	1	version = 1
3	1	type : télémétrie 1 / heartbeat 2
4	2	deviceId : uint16
6	4	seq : uint32
10	4	uptimeMs : uint32
14	1	count
15	3 × count	metricId : uint8 / value : int16
15 + 3 × count	1	checksum XOR

50 48 01 02 00 07 00 00 00 01 00 00 03 E8 00 F6

Étape de traitement (annexe B)


```
{
  "id": "temperature-mean",
  "type": "aggregate",
  "label": "Moy. 10 s",
  "metricId": 1,
  "operation": "mean",
  "windowMs": 10000,
  "maxSamples": 256
}
```

Type déjà disponible

Configurer une instance dans le JSON.

Nouveau type

Implémenter une factory locale, puis l’enregistrer dans stepFactories.

Filtres avant calculs. Fenêtres bornées. Validation au démarrage.

Commandes de démonstration (annexe C)

Terminal	Commande	Accès
1	npm run hub	127.0.0.1:3000
2	npm run demo:interactive	127.0.0.1:3100
3	npm run cli	Interface CLI

Défauts de transport

Arrêter le simulateur actif avec Ctrl+C, puis lancer npm run demo.

Validation : npm run check